

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Văn A

TIÊU ĐỀ KHOÁ LUẬN TỐT NGHIỆP

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



NGUYỄN VĂN A

TIÊU ĐỀ KHOÁ LUẬN TỐT NGHIỆP

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: TS. Nguyễn Văn B

Cán bộ đồng hướng dẫn: TS. Nguyễn Văn C

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Nguyen Van A

THESIS TITLE

BACHELOR'S THESIS

Major: Information Technology

Supervisor: Dr. Nguyen Van B

Co-Supervisor: Dr. Nguyen Van C

HANOI - 2026

Lời cảm ơn

Lời cam đoan

Tôi là ..., sinh viên lớp. Tôi xin cam đoan ...

Hà Nội, ngày tháng năm

Sinh viên

Nguyễn Văn A

Tóm tắt

Tóm tắt nội dung KL tốt nghiệp: Bài toán, phương pháp giải quyết, kết quả đạt được

Từ khóa: Liệt kê 3-5 từ khoá liên quan

Abstract

Summary of the graduation thesis: The problem statement, the proposed approach, and the achieved results.

Keywords: Liệt kê 3-5 từ khoá liên quan

Mục lục

Danh sách hình vẽ

Danh sách bảng

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
BiLSTM	Bidirectional Long Short Term Memory	Mô hình bộ nhớ dài hạn ngắn hạn hai chiều
BO	Buffer Overflow	Lỗi tràn bộ đệm

Chương 1

Đặt vấn đề

1.1 Bối cảnh và lý do chọn đề tài

Trong kỷ nguyên công nghệ số và sự phát triển mạnh mẽ của các thiết bị di động thông minh, thói quen tiếp cận thông tin và giải trí của con người đã trải qua những thay đổi sâu sắc. Một trong số đó là sự chuyển dịch từ việc đọc sách, truyện tranh in ấn truyền thống sang các nền tảng kỹ thuật số. Đặc biệt, đối với thể loại truyện tranh (manga, manhwa, comic), sự phát triển của Internet đã kéo theo sự ra đời của hàng loạt các dịch vụ lưu trữ và phân phối nội dung trực tuyến. Người dùng ngày càng có nhu cầu tiếp cận với hàng ngàn bộ truyện từ khắp nơi trên thế giới một cách nhanh chóng, tiện lợi mọi lúc mọi nơi thông qua chiếc điện thoại di động của mình.

Tuy nhiên, thực tế trải nghiệm người dùng trên các nền tảng đọc truyện hiện nay vẫn tồn tại nhiều hạn chế. Việc truy cập qua trình duyệt web trên thiết bị di động thường gặp phải tình trạng tối ưu hóa giao diện kém, tốc độ tải trang chậm do tải quá nhiều thành phần không cần thiết (quảng cáo, script theo dõi), và thiếu khả năng lưu trữ ngoại tuyến (offline) một cách hiệu quả. Hơn nữa, sự phân mảnh của các nguồn truyện khiến người dùng gặp khó khăn trong việc quản lý tập trung thư viện cá nhân, theo dõi tiến độ đọc, và nhận thông báo khi có chương mới.

Nhận thấy những vấn đề thiết thực trên, việc phát triển một ứng dụng di động chuyên biệt dành cho nền tảng Android, tập trung vào trải nghiệm đọc truyện mượt mà, quản lý thư viện thông minh và tích hợp các công nghệ hiện đại là một hướng đi vô cùng cấp thiết và mang nhiều ý nghĩa thực tiễn. Đề tài "Xây dựng ứng dụng đọc truyện tranh trực tuyến trên nền tảng Android" (MyBooksLibrary) được lựa chọn nhằm cung cấp một giải pháp toàn diện, khắc phục các nhược điểm của việc đọc truyện trên trình duyệt, đồng thời ứng dụng các kiến trúc phần mềm tiên tiến như Clean Architecture, MVVM cùng hệ sinh thái Jetpack Compose để đảm bảo hiệu suất và khả năng mở rộng của phần mềm.

1.2 Mô tả bài toán

Bài toán đặt ra cho đề tài là thiết kế và phát triển một ứng dụng đọc truyện tranh trên nền tảng hệ điều hành Android, có khả năng kết nối với một nguồn cung cấp dữ liệu truyện tranh trực tuyến (cụ thể là MangaDex API) để truy xuất danh sách truyện, thông tin chi tiết và dữ liệu hình ảnh của các chương truyện.

- **Đầu vào của bài toán:** Dữ liệu JSON được cung cấp từ các RESTful API của hệ thống MangaDex, bao gồm thông tin metadata của truyện (tên, tác giả, thể loại, mô tả), danh sách các chương, và đường dẫn URL tới các trang ảnh truyện. Ngoài ra, đầu vào còn bao gồm các tương tác của người dùng trên giao diện ứng dụng (tìm kiếm, thêm vào thư viện, vuốt trang, đánh dấu chương đã đọc).
- **Đầu ra của bài toán:** Một ứng dụng Android hoàn chỉnh với giao diện trực quan, hiển thị nội dung truyện tranh sắc nét, tối ưu hoá cho nhiều kích thước màn hình khác nhau. Hệ thống phải cho phép người dùng đọc truyện trực tiếp, quản lý danh sách truyện yêu thích, thống kê lịch sử đọc, và hỗ trợ tải dữ liệu ảnh về thiết bị để xem ngoại tuyến.
- **Các giả định:** Thiết bị di động của người dùng chạy hệ điều hành Android có phiên bản SDK tối thiểu là 30. Để thực hiện các thao tác lấy dữ liệu mới và đồng bộ hoá đám mây, thiết bị cần được kết nối với Internet. Đối với các chương truyện đã tải xuống, ứng dụng hoàn toàn có thể hoạt động ngoại tuyến.

1.3 Mục tiêu và phạm vi nghiên cứu

1.3.1 Mục tiêu nghiên cứu

Mục tiêu cốt lõi của đề tài là xây dựng thành công ứng dụng MyBooksLibrary với giao diện hiện đại, dễ sử dụng, đáp ứng tối đa nhu cầu đọc truyện tranh trên thiết bị di động. Cụ thể, các mục tiêu kỹ thuật bao gồm:

- Nghiên cứu và áp dụng thành thạo ngôn ngữ Kotlin, kết hợp với bộ công cụ xây dựng giao diện Jetpack Compose.

- Áp dụng kiến trúc Clean Architecture và mô hình MVVM (Model-View-ViewModel) nhằm tạo ra một mã nguồn mạch lạc, dễ bảo trì và có khả năng mở rộng cao.
- Sử dụng Dependency Injection (thông qua thư viện Hilt) để giảm sự phụ thuộc giữa các thành phần trong hệ thống.
- Tích hợp các thư viện xử lý bất đồng bộ (Coroutines/Flow) và giao tiếp mạng (Retrofit, OkHttp) một cách hiệu quả để đảm bảo hiệu năng, tránh tình trạng giật lag trên giao diện người dùng.
- Phát triển cơ chế quản lý dữ liệu cục bộ bằng Room Database để quản lý thư viện cá nhân và lịch sử đọc, đồng thời tích hợp Firebase để hỗ trợ xác thực người dùng và đồng bộ hoá dữ liệu đám mây.

1.3.2 Phạm vi nghiên cứu

- **Về nền tảng:** Đề tài chỉ giới hạn phát triển cho hệ điều hành di động Android.
- **Về dữ liệu:** Ứng dụng hiện tại tích hợp và lấy dữ liệu trực tiếp từ nguồn mở MangaDex API. Đề tài không đi sâu vào việc tự xây dựng backend server chứa nội dung truyện mà chỉ đóng vai trò là một client tiêu thụ API.
- **Về tính năng:** Ứng dụng tập trung vào các tính năng cốt lõi của một phần mềm đọc truyện: Khám phá truyện, tìm kiếm, hiển thị chi tiết, giao diện đọc (reader) linh hoạt, quản lý thư viện lưu trữ (thêm, xóa, đánh dấu tiến trình), tải xuống offline và đồng bộ hóa cơ bản.

1.4 Cấu trúc báo cáo

Báo cáo được bố cục thành 4 chương chính nhằm trình bày chi tiết và logic quá trình nghiên cứu, phát triển ứng dụng:

- **Chương 1: Đặt vấn đề.** Giới thiệu tổng quan về bối cảnh, lý do chọn đề tài, mô tả bài toán cần giải quyết, cùng với mục tiêu và phạm vi nghiên cứu.
- **Chương 2: Các công nghệ và thư viện sử dụng.** Tổng hợp các kiến thức cơ sở về ngôn ngữ lập trình, kiến trúc hệ thống (Clean Architecture, MVVM),

và phân tích vai trò của các thư viện cốt lõi (Jetpack Compose, Room, Retrofit, Hilt, Coil) được áp dụng trong dự án.

- **Chương 3: Phân tích yêu cầu và thiết kế hệ thống.** Liệt kê các tính năng của phần mềm, xác định các tác nhân, thiết kế sơ đồ Use Case và xây dựng sơ đồ tuần tự cho các luồng hoạt động chính yếu.
- **Chương 4: Đánh giá và kiểm thử.** Đưa ra phương pháp kiểm thử (Unit test), quá trình đánh giá tính ổn định, hiệu năng của ứng dụng và tổng kết các kết quả đạt được.

Chương 2

Các công nghệ và thư viện sử dụng

Chương này trình bày chi tiết về các nền tảng, ngôn ngữ lập trình, kiến trúc hệ thống và các thư viện cốt lõi được áp dụng trong quá trình xây dựng ứng dụng MyBooksLibrary. Đối với mỗi công nghệ, cơ sở lý thuyết sẽ được giới thiệu kết hợp với các đoạn mã nguồn minh họa thực tế từ dự án.

2.1 Ngôn ngữ Kotlin và Jetpack Compose

2.1.1 Ngôn ngữ Kotlin

Kotlin là một ngôn ngữ lập trình kiểu tĩnh (statically typed) do JetBrains phát triển, hiện được Google công nhận là ngôn ngữ chính thức và ưu tiên hàng đầu (Kotlin-first) để phát triển ứng dụng Android. Kotlin giúp giảm thiểu mã rập khuôn (boilerplate code), tăng tính an toàn (đặc biệt là Null Safety) và hỗ trợ mạnh mẽ lập trình bất đồng bộ thông qua Coroutines.

2.1.2 Bộ công cụ Jetpack Compose

Jetpack Compose là bộ công cụ xây dựng giao diện người dùng (UI) hiện đại của Android theo mô hình khai báo (Declarative UI). Thay vì sử dụng XML để thiết kế giao diện như trước đây, lập trình viên sử dụng trực tiếp mã Kotlin để định nghĩa UI. Compose tự động cập nhật lại (recompose) giao diện khi trạng thái (state) thay đổi.



placeholder_compose.png

Hình 2.1: Mô hình hoạt động của Jetpack Compose

Đoạn mã sau trích xuất từ `DiscoverScreen.kt` minh họa cách định nghĩa một thành phần giao diện (Composable) để hiển thị danh sách truyện:

```
1 @Composable
2 fun DiscoverScreenContent(
3     modifier: Modifier = Modifier,
4     vm: DiscoverViewModel = hiltViewModel(),
5 ) {
6     val uiState by vm.uiState.collectAsStateWithLifecycle()
7
8     Scaffold(
9         topBar = { EditorialTopBar(...) },
10        containerColor = MaterialTheme.colorScheme.background,
11    ) { innerPadding ->
12        PullToRefreshBox(
13            isRefreshing = uiState.isLoading && uiState.items.isNotEmpty(),
```

```

14         onRefresh = { vm.loadDiscover() },
15         modifier = Modifier.padding(innerPadding)
16     ) {
17         // Hien thi danh sach truyen (LazyColumn / LazyVerticalGrid)
18     }
19 }
20 }

```

Hình 2.1: Định nghĩa giao diện màn hình Khám phá bằng Jetpack Compose

2.2 Kiến trúc Clean Architecture và mô hình MVVM

2.2.1 Clean Architecture

Clean Architecture là một mẫu kiến trúc phần mềm do Robert C. Martin (Uncle Bob) đề xuất, nhằm chia nhỏ hệ thống thành các tầng (layer) độc lập với nhau, lấy các quy tắc nghiệp vụ (Domain/Business logic) làm trung tâm. Trong dự án, ứng dụng được chia thành 3 tầng chính:

- **Domain Layer:** Chứa các định nghĩa Use Case, Model. Không phụ thuộc vào Android framework.
- **Data Layer:** Chứa Repository và Data Source (API, Database).
- **Presentation (UI) Layer:** Chứa giao diện (Compose) và ViewModel.



placeholder_clean_architecture.png

Hình 2.2: Sơ đồ kiến trúc Clean Architecture kết hợp MVVM

2.2.2 Mô hình MVVM (Model - View - ViewModel)

Mô hình MVVM được sử dụng ở tầng Presentation. **ViewModel** đóng vai trò giữ và quản lý dữ liệu cho **View** (UI), đồng thời xử lý các tương tác của người dùng để gọi xuống tầng **Domain/Data**. Trong Kotlin, ViewModel sử dụng **StateFlow** để phát (emit) dữ liệu lên View một cách phản ứng (reactive).

```
1 @HiltViewModel
2 class MangaDetailViewModel @Inject constructor(
3     private val mangaRepository: MangaRepository,
4     private val libraryRepository: LibraryRepository,
5 ) : ViewModel() {
6
7     private val _uiState = MutableStateFlow(MangaDetailUiState())
8     val uiState: StateFlow<MangaDetailUiState> = _uiState.asStateFlow()
9 }
```

```

10     private fun loadMangaDetail(mangaId: String) {
11         viewModelScope.launch(ioDispatcher) {
12             mangaRepository.getMangaDetail(mangaId).onSuccess { manga ->
13                 _uiState.update { it.copy(mangaDetail = manga) }
14             }
15         }
16     }
17 }

```

Hình 2.2: Khởi tạo ViewModel và quản lý StateFlow trong MangaDetailViewModel.kt

2.3 Dependency Injection với Hilt

Hilt là một thư viện tiêm phụ thuộc (Dependency Injection - DI) dành riêng cho Android, được xây dựng dựa trên nền tảng Dagger. Hilt giúp giảm thiểu các đoạn mã khởi tạo lặp đi lặp lại bằng cách tự động cung cấp các đối tượng (instance) cần thiết vào các class, ví dụ như tự động tiêm `Repository` vào `ViewModel`.



Hình 2.3: Cơ chế Tiêm phụ thuộc của Hilt trong Android

Bằng cách sử dụng các annotation như `@HiltViewModel` và `@Inject`, Hilt sẽ tự động khởi tạo và tiêm `MangaRepository` vào `MangaDetailViewModel` mà không cần lập trình viên phải gọi hàm khởi tạo (như đã thấy ở đoạn mã trên).

2.4 Cơ sở dữ liệu cục bộ với Room Database

Room là một thư viện ORM (Object-Relational Mapping) nằm trong hệ sinh thái Android Jetpack, đóng vai trò như một lớp trừu tượng bao bọc SQLite. Room giúp thao tác với cơ sở dữ liệu bằng cách sử dụng các đối tượng Java/Kotlin thay vì viết câu lệnh SQL thuần túy, đồng thời kiểm tra lỗi SQL ngay lúc biên dịch.

Trong ứng dụng, Room được sử dụng để lưu trữ danh sách truyện yêu thích và tiến trình đọc của người dùng. Các thao tác truy xuất dữ liệu được định nghĩa thông qua các Data Access Object (DAO).



Hình 2.4: Các thành phần cốt lõi của Room Database

```
1 @Dao
2 interface LibraryDao {
3     @Upsert
4     suspend fun upsert(item: LibraryItemEntity)
5
6     @Query("SELECT * FROM library_items WHERE sync_status != 'PENDING_DELETE'")
7     fun observeAll(): Flow<List<LibraryItemEntity>>
8
9     @Query("UPDATE library_items SET is_favorite = :isFavorite WHERE manga_id = :mangaId")
10    suspend fun updateFavorite(mangaId: String, isFavorite: Boolean): Int
11 }
```

Hình 2.3: Định nghĩa LibraryDao.kt sử dụng Room

2.5 Giao tiếp mạng với Retrofit và Coroutines

Retrofit là một Client HTTP an toàn về kiểu (type-safe) phổ biến nhất cho Android. Retrofit cho phép định nghĩa các REST API thông qua các Interface và Annotation. Trong dự án này, Retrofit được kết hợp với Kotlin Coroutines (`suspend fun`) và Kotlinx Serialization để gọi API MangaDex một cách bất đồng bộ và tự động chuyển đổi chuỗi JSON trả về thành các đối tượng Kotlin.



Hình 2.5: Sơ đồ luồng giao tiếp HTTP qua Retrofit

```
1 interface MangaDexApi {
2     @GET("manga")
3     suspend fun getMangaList(
4         @Query("limit") limit: Int,
5         @Query("offset") offset: Int,
6         @Query("includes[]") includes: List<String> = listOf("cover_art"),
7     ): MangaListResponseDto
8
9     @GET("manga/{mangaId}")
10    suspend fun getMangaDetail(
```

```

11     @Path("mangaId") mangaId: String,
12     @Query("includes[]") includes: List<String> = listOf("cover_art"),
13 ): MangaDetailResponseDto
14 }

```

Hình 2.4: Sử dụng Retrofit định nghĩa các Endpoint trong MangaDexApi.kt

Ngoài ra, dự án sử dụng thư viện **Coil** (Coroutine Image Loader) được tích hợp sâu với Jetpack Compose thông qua hàm **AsyncImage** để tự động tải, giải mã và hiển thị hình ảnh (ảnh bìa truyện, trang truyện) trực tiếp từ các URL trả về của Retrofit.

2.6 Công nghệ Kiểm thử và Tích hợp liên tục (CI/CD)

Để đảm bảo chất lượng phần mềm và tự động hóa quy trình phát triển, dự án áp dụng các công cụ kiểm thử hiện đại cùng hệ thống Tích hợp liên tục (Continuous Integration).

2.6.1 Framework Kiểm thử

Dự án chú trọng đến việc viết Unit Test cho tầng logic nghiệp vụ nhằm hạn chế rủi ro lỗi trong quá trình mở rộng tính năng. Các thư viện kiểm thử cốt lõi bao gồm:

- **JUnit:** Framework kiểm thử tiêu chuẩn trên nền tảng Java/Kotlin, giúp thực thi các test case nhanh chóng trên môi trường máy ảo JVM.
- **MockK:** Thư viện giả lập (mocking) mạnh mẽ dành riêng cho Kotlin. MockK được dùng để tạo các đối tượng giả cho tầng **Repository** hay **DataStore**, qua đó cô lập **ViewModel** khỏi các sự phụ thuộc bên ngoài khi tiến hành kiểm thử.
- **Coroutines Test:** Cung cấp các công cụ như **runTest** và **TestDispatcher** để kiểm thử an toàn các tác vụ bất đồng bộ (suspend functions).

2.6.2 Tích hợp liên tục với GitHub Actions

GitHub Actions được lựa chọn làm hệ thống CI/CD do tính tiện dụng và sự gắn kết chặt chẽ với kho lưu trữ mã nguồn GitHub. Mỗi khi lập trình viên thực hiện

thao tác đẩy mã (push) hoặc tạo yêu cầu gộp mã (Pull Request) vào nhánh chính, hệ thống sẽ tự động kích hoạt luồng quy trình (workflow).

Luồng này đảm nhiệm các công việc quan trọng như: chạy kiểm tra chuẩn mực mã (Lint, Detekt), thực thi toàn bộ Unit Test, đo lường độ bao phủ mã (Code Coverage), và cuối cùng là tự động biên dịch ra tệp tin cài đặt APK. Nhờ vậy, dự án luôn duy trì được tính ổn định và ngăn ngừa các đoạn mã lỗi hòa trộn vào sản phẩm.

Dưới đây là một trích đoạn từ file cấu hình `ci.yml` tiêu biểu:

```
1 name: CI
2 on:
3   push:
4     branches: [main]
5   pull_request:
6     branches: [main]
7
8 jobs:
9   build-test:
10    runs-on: ubuntu-latest
11    steps:
12      - uses: actions/checkout@v4
13      - name: Set up JDK 21
14        uses: actions/setup-java@v5
15        with:
16          distribution: temurin
17          java-version: '21'
18      - name: Unit test and APK builds
19        run: ./gradlew testDebugUnitTest assembleDebug
```

Hình 2.5: Trích đoạn cấu hình GitHub Actions (`ci.yml`) chạy tự động kiểm thử và build ứng dụng

Chương 3

Phân tích yêu cầu và thiết kế hệ thống

Chương này đi sâu vào việc phân tích các yêu cầu của ứng dụng MyBooksLibrary, từ đó xây dựng các biểu đồ Use Case và biểu đồ tuần tự nhằm mô hình hóa hành vi và luồng hoạt động của phần mềm.

3.1 Phân tích yêu cầu

3.1.1 Yêu cầu chức năng

Ứng dụng cần đáp ứng các chức năng cốt lõi sau để đảm bảo trải nghiệm đọc truyện trọn vẹn:

- **Khám phá và tìm kiếm:** Hệ thống cho phép người dùng xem danh sách các truyện mới cập nhật, truyện phổ biến và tìm kiếm truyện theo từ khóa, thể loại hoặc tác giả.
- **Xem chi tiết và đọc truyện:** Hiển thị đầy đủ thông tin metadata của truyện (tóm tắt, đánh giá, trạng thái). Trình đọc (Reader) hỗ trợ tải trang truyện mượt mà, cho phép vuốt đọc hoặc ngang để chuyển trang.
- **Quản lý thư viện cá nhân:** Người dùng có thể lưu (bookmark) truyện yêu thích vào thư viện để dễ dàng theo dõi. Hệ thống tự động ghi nhận chương đang đọc dở để người dùng tiếp tục đọc ở lần mở ứng dụng sau.
- **Tải xuống ngoại tuyến:** Hỗ trợ tải các chương truyện về bộ nhớ cục bộ của thiết bị để có thể đọc khi không có kết nối mạng.
- **Đồng bộ dữ liệu:** Cho phép đăng nhập qua tài khoản Google để sao lưu và đồng bộ hóa thư viện cá nhân, lịch sử đọc lên máy chủ đám mây.

3.1.2 Yêu cầu phi chức năng

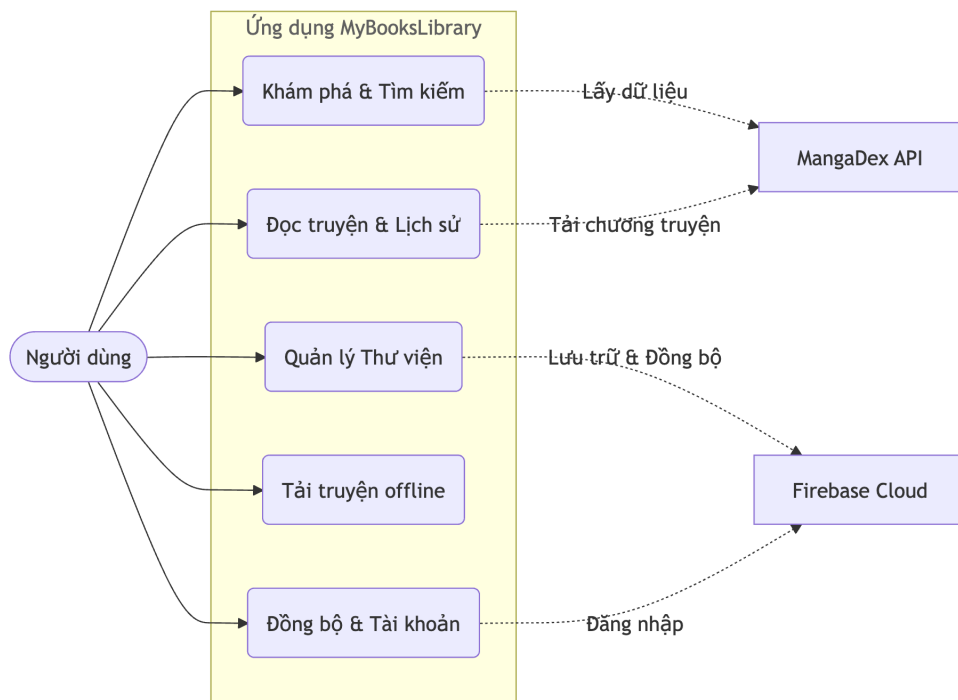
Bên cạnh các tính năng nghiệp vụ, hệ thống cần đảm bảo các tiêu chuẩn kỹ thuật sau:

- **Hiệu năng:** Thời gian tải màn hình danh sách và nội dung trang truyện không được vượt quá 3 giây trong điều kiện mạng ổn định. Giao diện vuốt trang phải đạt 60 FPS (khung hình/giây).
- **Khả năng tương thích:** Ứng dụng phải hoạt động ổn định trên các thiết bị Android từ API level 30 (Android 11) trở lên, tự động điều chỉnh giao diện phù hợp với nhiều kích thước màn hình (điện thoại, máy tính bảng).
- **Tính khả dụng ngoại tuyến:** Các truyện đã tải xuống phải được truy xuất và đọc mượt mà ngay cả khi thiết bị ngắt kết nối Internet hoàn toàn.
- **Kiến trúc mã nguồn:** Mã nguồn phải tuân thủ nghiêm ngặt nguyên lý Clean Architecture và SOLID, giúp dễ dàng bảo trì và mở rộng tính năng sau này.

3.2 Thiết kế hệ thống chi tiết

3.2.1 Sơ đồ Use Case tổng quát

Sơ đồ Use Case thể hiện sự tương tác giữa người dùng (Người đọc) và hệ thống ứng dụng, cũng như sự giao tiếp của ứng dụng với các hệ thống bên ngoài (MangaDex API, Firebase).



Hình 3.1: Sơ đồ Use Case tổng quát của ứng dụng MyBooksLibrary

3.2.2 Đặc tả chi tiết các Use Case quan trọng

Sau đây là đặc tả chi tiết cho 3 Use Case cốt lõi nhất của hệ thống, được trình bày thông qua bảng đặc tả.

Bảng 3.1: Đặc tả Use Case: Khám phá và Tìm kiếm truyện

Thuộc tính	Mô tả
Mã UC	UC-01
Tên Use Case	Khám phá và Tìm kiếm truyện
Tác nhân	Người dùng, MangaDex API
Mô tả	Cho phép người dùng duyệt qua danh sách các truyện thịnh hành, truyện mới cập nhật hoặc tìm kiếm truyện theo tên và bộ lọc.
Tiền điều kiện	Ứng dụng đã khởi động và có kết nối Internet ổn định.
Hậu điều kiện	Danh sách các bộ truyện (ảnh bìa, tên truyện) được hiển thị chính xác lên giao diện.
Luồng cơ bản	1. Người dùng mở ứng dụng, truy cập vào tab Khám phá hoặc Tìm kiếm. 2. Người dùng nhập từ khóa hoặc chọn các bộ lọc (nếu có). 3. Hệ thống gửi yêu cầu HTTP GET đến MangaDex API với các tham số tương ứng. 4. API trả về chuỗi JSON chứa danh sách truyện. 5. Hệ thống phân tích chuỗi JSON, lưu vào bộ đệm và hiển thị lên giao diện Compose.

Bảng 3.2: Đặc tả Use Case: Xem chi tiết và Đọc truyện

Thuộc tính	Mô tả
Mã UC	UC-02
Tên Use Case	Xem chi tiết và Đọc truyện
Tác nhân	Người dùng, MangaDex API
Mô tả	Người dùng chọn một truyện để xem thông tin chi tiết và truy cập vào giao diện đọc truyện (Reader) để duyệt qua các trang ảnh.
Tiền điều kiện	Người dùng đã chọn được một bộ truyện từ danh sách.
Hậu điều kiện	Cập nhật trạng thái và tiến độ đọc (chương số mấy, trang số mấy) vào cơ sở dữ liệu cục bộ.
Luồng cơ bản	1. Người dùng nhấn vào một bộ truyện. 2. Hệ thống gọi API để lấy thông tin chi tiết và danh sách các chương, hiển thị màn hình Chi tiết truyện. 3. Người dùng chọn một chương để đọc. 4. Hệ thống tải đường dẫn ảnh của chương đó qua At-Home API và hiển thị màn hình Đọc truyện. 5. Người dùng vuốt màn hình để chuyển trang. Coil tự động tải và hiển thị hình ảnh. 6. Khi người dùng thoát, hệ thống tự động lưu lại vị trí trang hiện tại vào Room Database.

Bảng 3.3: Đặc tả Use Case: Quản lý Thư viện cá nhân

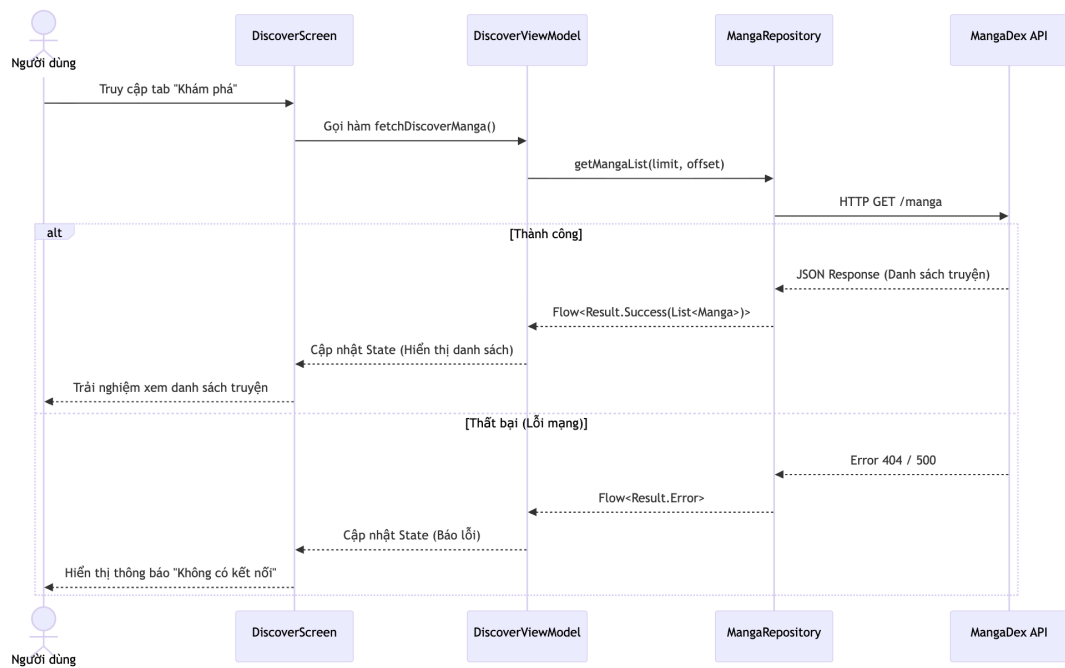
Thuộc tính	Mô tả
Mã UC	UC-03
Tên Use Case	Quản lý Thư viện cá nhân (Lưu truyện)
Tác nhân	Người dùng
Mô tả	Cho phép người dùng thêm các truyện đang theo dõi vào thư viện cá nhân để dễ dàng truy cập và nhận thông báo cập nhật.
Tiền điều kiện	Đang ở màn hình Chi tiết truyện.
Hậu điều kiện	Truyện được lưu thành công vào cơ sở dữ liệu Room cục bộ với trạng thái "Yêu thích".
Luồng cơ bản	1. Ở màn hình Chi tiết truyện, người dùng nhấn nút "Thêm vào Thư viện". 2. ViewModel nhận sự kiện và gọi xuống tầng Repository. 3. Repository thực hiện câu lệnh Insert/Update dữ liệu (Entity) của truyện vào bảng trong Room Database. 4. Flow theo dõi cơ sở dữ liệu phát ra tín hiệu thay đổi dữ liệu. 5. Giao diện tự động cập nhật, biểu tượng chuyển sang trạng thái "Đã lưu".

3.2.3 Luồng hoạt động chính (Sequence Diagrams)

Để hiểu rõ hơn về cách các thành phần bên trong hệ thống (theo kiến trúc Clean Architecture) tương tác với nhau, phần này trình bày chi tiết luồng hoạt động của hai quy trình cốt lõi bằng biểu đồ tuần tự.

Luồng lấy danh sách truyện từ MangaDex API

Luồng hoạt động này mô tả cách dữ liệu di chuyển từ lúc người dùng tương tác với giao diện cho đến khi dữ liệu từ máy chủ bên ngoài được đưa về và hiển thị.

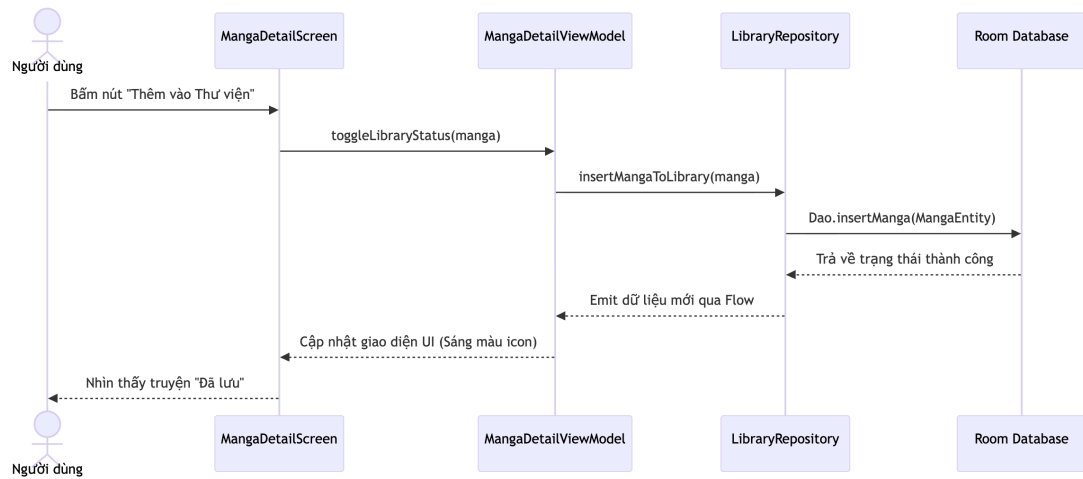


Hình 3.2: Sơ đồ tuần tự: Luồng lấy danh sách truyện từ MangaDex API

Giải thích chi tiết: Khi người dùng truy cập tab Khám phá, tầng UI (**DiscoverScreen**) kích hoạt hàm `fetchDiscoverManga()` trong **DiscoverViewModel**. **ViewModel** này đóng vai trò xử lý logic Presentation, nó gọi xuống **MangaRepository** ở tầng Data để yêu cầu danh sách truyện. **Repository** sử dụng **MangaDexApi** (Retrofit) để thực hiện một HTTP GET request đến máy chủ. Nếu thành công, chuỗi JSON trả về được parse thành các Object **MangaListResponseDto**. **Repository** chuyển đổi Dto này thành **MangaModel** của tầng Domain và gửi kết quả về **ViewModel** thông qua `Flow<Result.Success>`. Cuối cùng, **ViewModel** cập nhật UI State, và Jetpack Compose phản ứng lại bằng cách recompose để hiển thị danh sách truyện lên màn hình. Trong trường hợp lỗi mạng, một `Result.Error` sẽ được trả về để hiển thị thông báo lỗi.

Luồng lưu truyện vào thư viện local bằng Room

Luồng này mô tả thao tác lưu trữ dữ liệu hoàn toàn ở bộ nhớ cục bộ, bảo đảm tính tức thời và cho phép ứng dụng phản hồi ngay lập tức cho người dùng.



Hình 3.3: Sơ đồ tuần tự: Luồng lưu truyện vào thư viện cá nhân

Giải thích chi tiết: Từ màn hình chi tiết, khi người dùng nhấn nút "Thêm vào Thư viện", `MangaDetailScreen` gửi một sự kiện `toggleLibraryStatus()` tới `MangaDetailViewModel`. Nhận được tín hiệu, `ViewModel` gọi phương thức `insertMangaToLibrary()` của `LibraryRepository`. `Repository` sẽ tạo một `LibraryItemEntity` và dùng `LibraryDao` để thực thi truy vấn `@Upsert` vào `Room Database`. `Room Database` xử lý lưu trữ vào `SQLite` thành công. Do `ViewModel` luôn lắng nghe dữ liệu từ hàm `observeAll()` (trả về kiểu `Flow`), việc thay đổi dữ liệu trong database lập tức kích hoạt luồng phát ra dữ liệu mới. Trạng thái `isFavorite` được cập nhật, và giao diện UI tự động thay đổi (đổi màu biểu tượng trái tim) để báo hiệu thao tác lưu trữ thành công.

Chương 4

Kiểm thử và Đánh giá hệ thống

Chương này trình bày chi tiết về chiến lược kiểm thử tự động, cấu hình quy trình tích hợp liên tục (CI/CD) và kết quả của quá trình kiểm thử thủ công. Đồng thời, chương cũng đưa ra những đánh giá khách quan về ưu, nhược điểm của phần mềm và đề xuất các hướng phát triển trong tương lai.

4.1 Kiểm thử tự động (Unit Testing)

4.1.1 Chiến lược kiểm thử trong Clean Architecture

Với việc áp dụng Clean Architecture, hệ thống được chia nhỏ thành các tầng riêng biệt, tạo điều kiện thuận lợi cho việc kiểm thử tự động. Chiến lược kiểm thử chính trong dự án tập trung vào:

- **Tầng Domain và Data:** Kiểm thử các logic cốt lõi như Use Case, DAO thông qua JUnit. Tầng Data thường kết hợp với thư viện giả lập (Mocking) hoặc Robolectric để kiểm thử các tác vụ truy xuất dữ liệu độc lập với nền tảng Android phần cứng.
- **Tầng Presentation (UI):** Trọng tâm đặt vào việc kiểm thử các ViewModel. Bằng cách sử dụng thư viện MockK, toàn bộ các phụ thuộc (Dependencies) như Repository hay DataStore đều được giả lập, giúp cô lập hoàn toàn logic của ViewModel.

4.1.2 Kịch bản kiểm thử ViewModel tiêu biểu

Đoạn mã dưới đây là một kịch bản kiểm thử (Test case) thực tế trích từ MangaDetailViewModel. Test case này nhằm xác minh tính đúng đắn của logic xử lý yêu cầu tải xuống một chương truyện ngoại tuyến.

```
1 @Test
2 fun startChapterDownload_delegatesToOfflineDownloadManager() = runTest(
    mainDispatcherRule.dispatcher.scheduler) {
```

```

3 // 1. Arrange (Chuan bi)
4 val manager = mockManager()
5 val viewModel = createViewModel(manager)
6 advanceUntilIdle()
7
8 // 2. Act (Hanh dong)
9 viewModel.startChapterDownload(CHAPTER_ID)
10 advanceUntilIdle()
11
12 // 3. Assert (Kiem tra/Xac nhan)
13 coVerify(exactly = 1) { manager.enqueueDownload(MANGA_ID, CHAPTER_ID) }
14 }

```

Hình 4.1: Trích đoạn Unit Test áp dụng mẫu Arrange - Act - Assert

Giải thích quy trình (Arrange - Act - Assert):

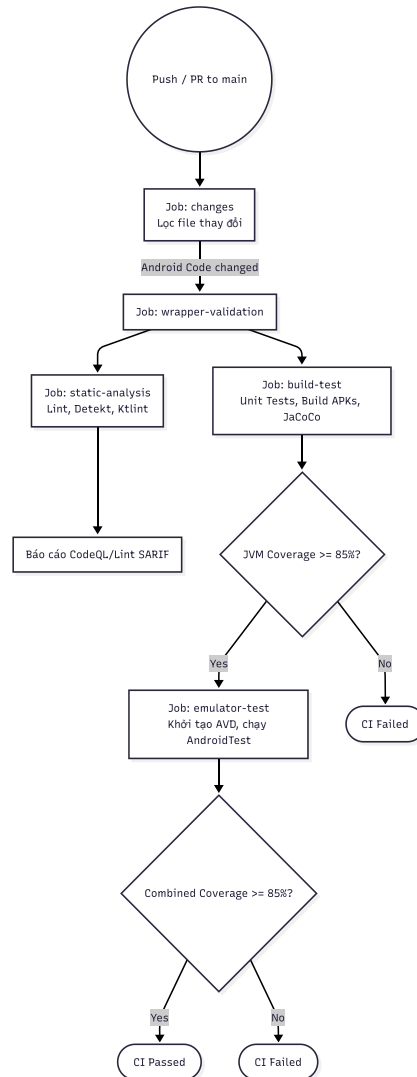
- **Arrange (Chuẩn bị):** Hệ thống khởi tạo một `ViewModel` cùng với một `OfflineDownloadManager` đã được cấu hình giả lập (mock). Luồng thời gian coroutines được giả lập thông qua `advanceUntilIdle()` để các tiến trình bất đồng bộ khởi tạo ban đầu hoàn tất.
- **Act (Hành động):** Kích hoạt phương thức `startChapterDownload()` mô phỏng việc người dùng thao tác nhấn nút "Tải xuống" trên giao diện.
- **Assert (Xác nhận):** Sử dụng hàm `coVerify` của `MockK` để kiểm chứng xem `ViewModel` có thực sự giao phó (delegate) nhiệm vụ bằng cách gọi đúng phương thức `enqueueDownload` của trình quản lý tải xuống (manager) với tham số chính xác hay không. Nếu phương thức được gọi đúng 1 lần (`exactly = 1`), bài kiểm thử đạt yêu cầu (Pass).

4.2 Tích hợp liên tục (CI) với GitHub Actions

Để hiện thực hóa việc tự động hóa kiểm thử, dự án áp dụng hệ thống CI/CD trên nền tảng GitHub Actions.

4.2.1 Luồng quy trình CI tự động

Mỗi khi lập trình viên tạo một thao tác đẩy mã (Push) hoặc tạo Yêu cầu gộp mã (Pull Request) lên nhánh `main`, GitHub Actions sẽ tự động kích hoạt luồng CI thông qua tệp cấu hình `ci.yml`.



Hình 4.1: Sơ đồ quy trình Tích hợp liên tục (CI/CD) tự động qua GitHub Actions

Quá trình này bao gồm các tác vụ chính:

1. **Lọc thay đổi (changes):** Phân tích mã nguồn để quyết định việc chạy các khâu kiểm thử phù hợp nhằm tối ưu hiệu suất máy chủ lưu trữ.
2. **Phân tích tĩnh (static-analysis):** Quét toàn bộ mã nguồn bằng Android Lint và Detekt để đảm bảo cấu trúc và chất lượng mã đáp ứng tiêu chuẩn.

3. **Biên dịch và Unit Test (build-test):** Tự động biên dịch ứng dụng ra định dạng APK và chạy toàn bộ Unit test. Sau đó, tiến hành đo lường độ phủ mã (Code Coverage). Nếu tỷ lệ bao phủ dưới mức 85%, tiến trình sẽ tự động bị hủy bỏ và báo lỗi (Fail).
4. **Kiểm thử giao diện (emulator-test):** Tự động khởi tạo máy ảo Android (AVD) ngay trên server để chạy các bài test giao diện thực tế liên quan đến vòng đời UI.

Nhờ sự giám sát chặt chẽ và tự động hóa này, ứng dụng đảm bảo không có đoạn mã lỗi nào vô tình lọt vào luồng chính, tiết kiệm đáng kể thời gian rà soát thủ công.



figures/github_actions_success.png

Hình 4.2: Kết quả chạy GitHub Actions thành công trên giao diện web

4.3 Kiểm thử thủ công (Manual Testing)

Quá trình kiểm thử được thực hiện trực tiếp trên thiết bị di động thật (chạy hệ điều hành Android) nhằm đánh giá trải nghiệm thực tế của người dùng. Các bảng dưới đây trình bày kịch bản và kết quả kiểm thử cho 4 tính năng quan trọng nhất.

4.3.1 Kiểm thử chức năng Khám phá và Tìm kiếm

Bảng 4.1: Kịch bản kiểm thử: Khám phá và Tìm kiếm truyện

ID	Chức năng	Các bước thực hiện	Kết quả mong đợi	Kết quả
TC-01	Tải danh sách truyện	Mở ứng dụng, chọn tab Khám phá.	Danh sách truyện hiển thị đầy đủ ảnh bìa và tên truyện nhanh chóng.	Pass
TC-02	Tìm kiếm truyện	Nhập từ khóa vào thanh tìm kiếm.	Hiển thị đúng các bộ truyện có chứa từ khóa vừa nhập.	Pass
TC-03	Lọc theo thể loại	Mở bộ lọc, chọn các thể loại tùy ý.	Chỉ hiển thị các bộ truyện thuộc các thể loại đã chọn.	Pass
TC-04	Cuộn trang vô tận	Cuộn xuống cuối danh sách truyện.	Tự động tải thêm (pagination) các bộ truyện tiếp theo mượt mà.	Pass

4.3.2 Kiểm thử chức năng Xem chi tiết và Đọc truyện

Bảng 4.2: Kịch bản kiểm thử: Xem chi tiết và Đọc truyện

ID	Chức năng	Các bước thực hiện	Kết quả mong đợi	Kết quả
TC-05	Tải chi tiết truyện	Bấm vào 1 truyện từ danh sách.	Hiện thông tin tác giả, tóm tắt, và danh sách các chương.	Pass
TC-06	Load trang truyện	Chọn Chương 1 để bắt đầu đọc.	Ảnh trang truyện được tải lên đầy đủ, không bị vỡ hạt.	Pass
TC-07	Thao tác chuyển trang	Vuốt ngang/dọc trên màn hình.	Chuyển sang trang tiếp theo mượt mà, không giật lag.	Pass
TC-08	Lưu lịch sử đọc	Thoát ra khỏi chương đang đọc dở.	Hệ thống tự động ghi nhận chương và trang dừng lại cuối cùng.	Pass

4.3.3 Kiểm thử chức năng Quản lý Thư viện

Bảng 4.3: Kịch bản kiểm thử: Quản lý Thư viện cá nhân

ID	Chức năng	Các bước thực hiện	Kết quả mong đợi	Kết quả
TC-09	Thêm vào Thư viện	Bấm nút "Lưu truyện" ở màn hình Chi tiết.	Biểu tượng đổi màu, truyện xuất hiện trong tab Thư viện.	Pass
TC-10	Xóa khỏi Thư viện	Bấm lại nút "Lưu truyện" (bỏ lưu).	Biểu tượng trở lại ban đầu, truyện biến mất khỏi tab Thư viện.	Pass
TC-11	Đồng bộ hóa đám mây	Đăng nhập Google, nhấn đồng bộ.	Lịch sử đọc và thư viện được đẩy lên/tải về từ Firebase.	Pass

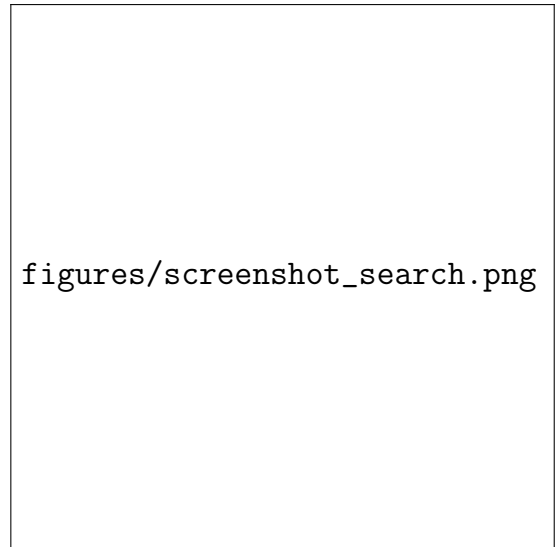
4.3.4 Kiểm thử chức năng Tải xuống (Offline)

Bảng 4.4: Kịch bản kiểm thử: Tải truyện ngoại tuyến

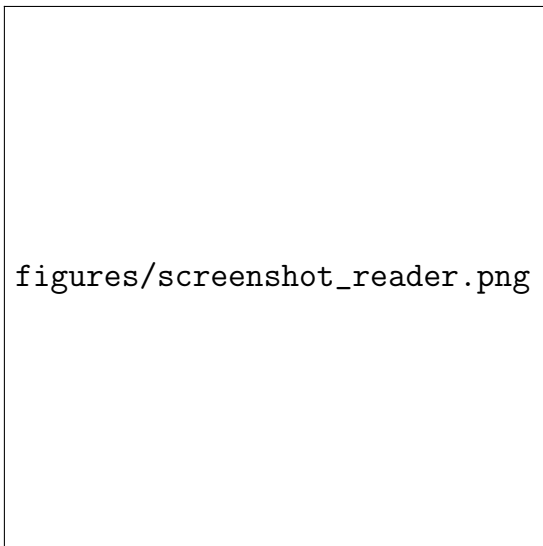
ID	Chức năng	Các bước thực hiện	Kết quả mong đợi	Kết quả
TC-12	Bắt đầu tải chương	Nhấn icon "Tải xuống" cạnh tên chương.	Thanh tiến trình xuất hiện, hiển thị % hoàn thành.	Pass
TC-13	Đọc không cần mạng	Tắt Wi-Fi/4G, bấm vào chương đã tải.	Truyện vẫn mở lên và lướt qua được tất cả các trang bình thường.	Pass
TC-14	Quản lý dung lượng	Vào phần Quản lý tải xuống, xóa chương.	Dung lượng bộ nhớ được giải phóng, xóa ảnh khỏi thiết bị.	Pass

4.4 Kết quả kiểm thử trên giao diện thực tế

Sau khi thực hiện các kịch bản kiểm thử, hệ thống ghi nhận ứng dụng hoạt động ổn định và đáp ứng đúng các luồng nghiệp vụ đề ra. Dưới đây là các ảnh chụp màn hình (screenshot) minh họa kết quả hiển thị trên thiết bị di động.



Hình 4.3: Giao diện màn hình Khám phá (trái) và Tìm kiếm/Lọc truyện (phải)



Hình 4.4: Giao diện màn hình Đọc truyện (trái) và Thư viện cá nhân (phải)

4.5 Đánh giá ưu, nhược điểm của ứng dụng

4.5.1 Ưu điểm

- **Giao diện hiện đại, mượt mà:** Việc áp dụng Jetpack Compose và tiêu chuẩn Material Design 3 giúp ứng dụng có trải nghiệm tương tác (vuốt, chạm, chuyển cảnh) linh hoạt và đạt hiệu suất đồ họa cao.
- **Kiến trúc mở rộng tốt:** Tuân thủ Clean Architecture giúp tách biệt hoàn

toàn logic nghiệp vụ với UI. Việc này hỗ trợ lập trình viên dễ dàng nâng cấp, thay thế một API cung cấp truyện khác trong tương lai mà không làm vỡ các thành phần hiện tại.

- **Tối ưu hoá đọc ngoại tuyến:** Hệ thống quản lý dữ liệu Room kết hợp với luồng tải bất đồng bộ giúp lưu trữ hình ảnh vật lý hiệu quả, giải quyết được bài toán đọc truyện khi mất kết nối mạng.

4.5.2 Nhược điểm

- **Phụ thuộc nguồn dữ liệu duy nhất:** Ứng dụng hiện tại hoạt động dựa hoàn toàn vào API của MangaDex. Nếu hệ thống máy chủ này bảo trì hoặc thay đổi cấu trúc dữ liệu, toàn bộ ứng dụng sẽ bị tê liệt do không thể truy xuất nội dung.
- **Quản lý bộ nhớ Cache chưa triệt để:** Quá trình tải và hiển thị ảnh thông qua Coil sinh ra lượng lớn cache. Dù đem lại tốc độ mở truyện cực nhanh ở các lần đọc sau, nhưng nếu thiếu cơ chế dọn dẹp định kỳ thông minh, dung lượng ứng dụng có thể "phình" to ảnh hưởng tới thiết bị bộ nhớ thấp.
- **Thiếu tính năng tương tác xã hội:** Một ứng dụng đọc truyện lý tưởng cần có sự tương tác. Hiện tại người dùng chỉ có thể trải nghiệm cá nhân, chưa thể để lại bình luận, thảo luận hoặc chia sẻ trực tiếp với cộng đồng đọc giả.

4.6 Hướng phát triển tương lai

Để khắc phục những hạn chế hiện có và nâng tầm chất lượng sản phẩm, các hướng phát triển trong thời gian tới được đề xuất bao gồm:

- **Xây dựng kiến trúc Đa nguồn (Multi-source):** Bổ sung các module theo cơ chế Plugin để cho phép người dùng chuyển đổi lấy dữ liệu từ nhiều website/API truyện tranh khác nhau, giảm thiểu rủi ro khi một server nguồn bị sập.
- **Phát triển tính năng cộng đồng:** Xây dựng một Backend Server trung gian (Node.js/Spring Boot) hoặc mở rộng Firebase để hỗ trợ hệ thống bình

luận (Comment), diễn đàn thu nhỏ và tính năng đánh giá (Rating) cho từng bộ truyện.

- **Hệ thống gợi ý thông minh (Recommendation System):** Áp dụng các mô hình học máy (Machine Learning) cơ bản để phân tích sở thích, thói quen đọc truyện của từng cá nhân, từ đó tự động đưa ra các đề xuất truyện phù hợp.
- **Tối ưu hóa quản lý dung lượng tự động:** Tích hợp tính năng tự dọn dẹp các ảnh cache đã lưu quá 30 ngày, hoặc đề xuất người dùng tự xóa các chương truyện tải xuống sau khi đọc xong.

Kết luận

Tóm tắt bài toán, giải pháp và các kết quả đạt được